Ex NO: 4 Packet sniffing using Raw sockets in Python

Aim:

To implement a packet sniffer using raw sockets in python that captures live network traffic and parses Ethernet, IP, and transport-layer (TCP/UDP/ICMP) headers.

Theory:

Raw sockets: Normal sockets (SOCK_STREAM, SOCK_DGRAM) let an application send/receive data through the OS's protocol stack (TCP/UDP), hiding lower-layer headers. A raw socket (SOCK_RAW) bypasses this abstraction - it gives the program direct access to packets at the IP or even the ethernet (data link) layer, including all headers. This is why raw sockets are the basis of tools like wireshark, tcpdump, and custom sniffers.

Why root/admin privileges are required: Raw sockets can read/inject arbitrary traffic on the network, which is a security-sensitive capability. The OS restricts their creation to privileged (root/administrator)

Platform note: This lab is written for Linux (raw sockets work most completely there). On Windows, raw sockets have restrictions (no ethernet-layer capture, limited protocols); on macOS, similar restrictions apply. If you're on Windows, run this inside a Linux VM or WSL2 (with appropriate permissions) - this is the standard lab setup.

EX NO:4

Aim:

```
File  Edit  Format  Run  Options  Window  Help
import socket
import struct

# Create raw socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)

# Local machine IP
host = socket.gethostbyname(socket.gethostname())

sniffer.bind((host, 0))

# Include IP headers
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# Enable promiscuous mode (Windows)
sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("Packet Sniffer Started...\n")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65535)

        ip_header = raw_data[0:20]

        iph = struct.unpack('!BBHHHBBH4s4s', ip_header)

        version = iph[0] >> 4
        ihl = iph[0] & 0xF

        ttl = iph[5]
        protocol = iph[6]

        source = socket.inet_ntoa(iph[8])
        destination = socket.inet_ntoa(iph[9])

        print("----------------------------------------")
        print("Version      :", version)
        print("Header Length:", ihl * 4)
        print("TTL          :", ttl)
        print("Protocol     :", protocol)
```

```
        print("Source IP    :", source)
        print("Destination  :", destination)
        print("Packet Length:", len(raw_data))

except KeyboardInterrupt:
    sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    print("\nStopped")
```

```
Protocol      : 17
Source IP     : 172.16.8.7
Destination   : 224.0.0.251
Packet Length: 157
----------------------------------------
Version       : 4
Header Length: 20
TTL           : 62
Protocol      : 17
Source IP     : 142.251.221.174
Destination   : 172.16.10.41
Packet Length: 1234
----------------------------------------
Version       : 4
Header Length: 20
TTL           : 128
Protocol      : 17
Source IP     : 172.16.10.41
Destination   : 142.251.221.174
Packet Length: 67
----------------------------------------
Version       : 4
Header Length: 20
TTL           : 62
Protocol      : 17
Source IP     : 142.251.221.174
Destination   : 172.16.10.41
Packet Length: 54

Stopped
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd C:\Users\HDC0222016\Desktop\PacketSniffer

C:\Users\HDC0222016\Desktop\PacketSniffer>python sniffer.py
Packet Sniffer Started...

----------------------------------------
Version       : 4
Header Length: 20
TTL           : 1
Protocol      : 17
Source IP     : 172.16.9.69
Destination   : 239.255.255.250
Packet Length: 198
----------------------------------------
Version       : 4
Header Length: 20
TTL           : 1
Protocol      : 17
Source IP     : 172.16.8.156
Destination   : 224.0.0.251
Packet Length: 90
----------------------------------------
Version       : 4
Header Length: 20
TTL           : 1
Protocol      : 17
Source IP     : 172.16.8.156
```

Step1: Install python

windows

Download the latest python.

Result:

Thus the packet sniffing using raw sockets in python has been executed & verified.